

CMS RAN Demo Script (60 minutes)

Team: Sheyn (Tech Lead) | Nathan (Full Stack) | Sesham (DevSecOps) | Bob (Program Manager) **Live URL:** <https://dm3z85jabb0r5.cloudfront.net> **Repo:** <https://github.com/skywarditsolutions/cmsffmran>

Pre-Demo Setup (T-5 min)

- Two browser windows ready:
 - **Window A:** Consumer intake at `/assist` (blank form)
 - **Window B:** Agent dashboard logged in as `bob.amos@ran.demo` / `Agent#Demo123`
 - Admin dashboard open in a tab (`admin@ran.demo` / `Admin#Demo123`)
 - Have the GitHub repo open in a background tab for code references
 - CloudFront may cache; if changes look stale, hard-refresh (Cmd+Shift+R)
-

Opening (2 min) — Bob

"Good morning. We're the Skyward team: I'm Bob, Program Manager; Sheyn is our Tech Lead; Nathan is our Full Stack Engineer; and Sesham leads DevSecOps. Over the past several weeks we built a working, AWS-native prototype of the CMS RAN that replaces the current Help On Demand system with a non-proprietary, fully transparent solution. Today we'll walk through all seven SOO questions with a live demonstration of the system in action."

DEMO - We demo the solution and all 3 screens. Sheyn drives, Bob narrates

We studied the current solution in depth. And believe we can offer a stronger alternative. Here is the prototype we came up

1. Submit a Request -> Wait for it to show how it gets routed. We will talk about routing logic later
2. Show Agent/Broker page. Go over left panel, right panel, configurable schedule, history, etc. We will add links to Training and other details as needed
3. Show Request being accepted. Routing logic is defined in Step Functions which we will cover next
4. Show Admin dashboard. Talk about analytics, history.... aka a Control Pane over your entire operations. You can see which agents are online. You can even reach out to them if needed.

Note, this is just a prototype that the team built in couple of days. Native apps for iOS and Android will be developed in couple of weeks. Now let's go deeper and look under the hood. There were 7 questions raised from you guys so will answer them next and show examples within a solution for each

Q1: Non-Proprietary Solution & Government Maintainability (7 min) — Sheyn

Talking Points

- "Every line of code, infrastructure, and documentation is in our GitHub repo. There are zero proprietary dependencies, zero vendor lock-in, and zero black-box components."

- "The entire stack uses open-source technologies: React, TypeScript, Node.js Lambda, DynamoDB, Step Functions, Cognito, API Gateway. All AWS-native, all Government-owned upon delivery."
- "Infrastructure is defined as code in Terraform. A Government team can reproduce the entire environment by running `terraform apply` with a different region parameter. No manual console clicks."
- "All documentation lives alongside the code: architecture diagrams, security compliance mapping, deployment scripts. The Government gets the repo and everything in it."

Live Demo Actions

1. **Show the GitHub repo** (github.com/skywarditsolutions/cmsffmran)
 - Scroll through the directory structure: `backend/`, `frontend/`, `infra/terraform/`, `docs/`, `scripts/`
 - Point out: `architecture.md`, `security-compliance.md`, `deploy.sh`
 - "This is the complete source. Nothing is hidden, nothing is compiled to a binary you can't inspect."
2. **Show the Terraform structure** (`infra/terraform/`)
 - `lambda.tf`, `dynamodb.tf`, `stepfunctions.tf`, `apigw_http.tf`, `apigw_ws.tf`, `cognito.tf`
 - "Every AWS resource is declared here. The Government can audit, modify, or reproduce any component."
3. **Show the deploy script** (`scripts/deploy.sh`)
 - "One command provisions the entire system from scratch: build, Terraform apply, seed, Cognito users, frontend upload. This IS the transition artifact."

Transition

"Now let's look at how the business rules and routing logic work, which is where the system really shines."

Q2: Business Rules, Routing Logic & Configurability (10 min) — Sheyn + Nathan

Talking Points

- "Routing is orchestrated by AWS Step Functions, a visual, auditable workflow. Every request submitted maps to one Execution run of the step Function. You can see every state, every transition, every timeout in the ASL definition file. AND this workflow is very easy to modify and test." **SHOW STEPFUNCTIONS in AWS here**
- "The matching algorithm is pure TypeScript (or can be implemented in any other language Python, JS, Java, etc.) in `backend/src/lib/matching.ts`. It's readable, testable, and modifiable by any Government engineer."
- "Business rules are NOT hardcoded. They live in a DynamoDB config table and are editable in real-time from the admin dashboard without a redeploy."
- "Eligibility filters: agent must be online, training current, not out of office, licensed in the consumer's state, speak the consumer's language, have capacity, and be within their availability"

window."

- "Ranking: load balancing is the dominant factor, then proximity, then fairness via least-recently-assigned. The proximity weight is admin-configurable."

Live Demo Actions

1. Show the admin dashboard (Window: admin)

- Point to the "Business rules" card: referral timeout (30s demo / 900s production), max routing attempts (5), proximity weight (0.3)
- "I can change any of these right now and the next request will use the new values instantly. No code change, no redeploy."
- Change the proximity weight from 0.3 to 0.5 and click Save

2. Show the routing state machine (GitHub: [infra/terraform/routing.asl.json.tftpl](#))

- "This is the ASL definition. The flow is: Match -> HasCandidate? -> Notify (wait for accept with timeout) -> on timeout, loop back to Match. After max attempts, fall to SafetyNet. After SafetyNet timeout, Queued."

3. Show the matching code (GitHub: [backend/src/lib/matching.ts](#))

- Walk through `isEligible()` and `rankCandidates()` briefly
- "The formula is on one line: `loadFactor * (1-w) + proximityFactor * w + recencyFactor * 0.1`. Any Government engineer can understand and modify this."

4. LIVE ROUTING DEMO: Submit a consumer request

- Switch to Window A (consumer intake at `/assist`)
- Fill in: First name "Demo", Last name "Test", Phone "5551234567", ZIP "48201", Language "French"
- Check TCPA consent, click "Request a callback"
- Note the Reference ID on the confirmation page
- Switch to Window B (Bob's agent dashboard)
- "Within seconds, the referral appeared on Bob's dashboard via real-time WebSocket push. He has 30 seconds to accept."
- Click "Accept" to show the PII reveal (name, phone, email, location)
- "Notice the consumer's personal information was hidden until Bob accepted. PII is encrypted at rest with KMS and only decrypted for the accepting agent."

Transition

"That consumer intake form is our integration point with HealthCare.gov. Let's talk about that."

Q3: HealthCare.gov Integration (7 min) — Nathan

Live Demo Actions

1. Show the consumer form (Window A)

- Point out: HealthCare.gov-style header and color palette. We use CMS Design System here so this can be changed very easily.
- The disclaimer block: "Before you continue..." with third-party and non-endorsement language
- TCPA consent checkbox (server rejects without it)
- ZIP code field with helper text "Used to match you with an agent licensed in your state"
- Language dropdown (English, Spanish, Vietnamese, Chinese, Tagalog, Korean, Creole, French, German). Again, this is just for the demo.

2. **Show the consumer status page** (already submitted in Q2)

- "The consumer sees 'Finding an available agent...' with their reference ID. They can keep browsing HealthCare.gov. When the agent accepts, this updates in real-time."

Talking Points

- "The consumer intake form at `/assist` is designed to be embedded as a widget within HealthCare.gov's existing pages. It's a self-contained React component that communicates via REST API."
- "The form collects exactly what the existing application has (but this can be easily expanded in the future): name, phone, phone type, preferred contact method, ZIP code, preferred language, and TCPA consent. It includes the third-party disclaimer, non-endorsement notice, and limited-use-of-information notice."
- "The API is a standard HTTP REST surface: `POST /requests` for submission, `GET /requests/{id}` for status. Authentication for consumer endpoints is unauthenticated (public intake), while agent and admin endpoints require Cognito JWT."
- "The consumer status page uses WebSocket to push live updates without page navigation. The consumer can stay on HealthCare.gov and see their status change in real-time."
- "For production integration: the form would sit behind HealthCare.gov's existing SSO/identity layer. We'd coordinate with the CMS Marketplace integration team on API contracts, callback URLs, and co-branding requirements. The form is already styled to match HealthCare.gov's visual language."

Transition

"Security and privacy are foundational to this design. Sesham will walk us through that."

Q4: Security & Privacy (7 min) — Sesham

Talking Points

- "We are Team Skyward. We run and maintain many systems at CMS in Production and know first hand what it takes to obtain and maintain an ATO. So we designed the system in a way that it is very ATO friendly"
- "Consumer PII is field-level encrypted with a KMS Customer Managed Key before it ever touches DynamoDB. The plaintext is never logged, never sent over WebSocket, and never visible to non-accepting agents."
- "PII is only decrypted in the `agentAccept` Lambda handler, and only after a successful accept. The response goes directly to the accepting agent over TLS. No other agent, no admin, no other system component sees the plaintext."

- "DynamoDB tables also have server-side encryption with KMS. KMS key rotation is enabled."
- "Identity and access: Cognito User Pools with role-based access control. Agents and admins are in separate Cognito groups. API Gateway JWT authorizer validates tokens on every protected request. Lambda handlers also check roles server-side as defense-in-depth."
- "Least-privilege IAM: each Lambda role has scoped permissions to only the DynamoDB tables, KMS keys, SNS topics, and SES identities it needs."
- "Hosting target is AWS GovCloud. The Terraform is region-parameterized. Changing to `us-gov-west-1` is a one-line variable change."
- "For ATO: we map to CMS ARS controls. CloudWatch logs on all Lambdas. Production would add structured audit events for PII access (who decrypted which record and when) and CloudTrail on KMS decrypt operations."

Live Demo Actions

1. **Show the security compliance doc** (GitHub: `docs/security-compliance.md`)
 - Scroll through the tables showing requirement-to-implementation mapping
 - "This is our ATO evidence starting point. Every CMS requirement is mapped to a specific code artifact."
2. **Show KMS encryption code** (GitHub: `backend/src/lib/crypto.ts`)
 - "This is the encrypt/decrypt module. PII is encrypted before DynamoDB put, decrypted only on accept."
3. **Show the auth code** (GitHub: `backend/src/lib/auth.ts`)
 - "getIdentity extracts role from Cognito JWT claims. Every admin handler checks `requireAdmin()` before processing."
4. **Show Terraform IAM** (GitHub: `infra/terraform/iam.tf`)
 - "Least-privilege policies. No wildcard permissions."
5. **Show the agent accept flow** (back to Window B where Bob accepted)
 - "When Bob clicked Accept, the server verified his identity, atomically claimed the request, decrypted the PII, and returned it only to him. If another agent tried to accept the same request, they'd get a 409 conflict."

Transition

"Now let's talk about how we get from prototype to production. Bob will cover the 90-day delivery plan."

Q5: 90-Day Delivery Plan (8 min) — Bob

Talking Points

- "Our plan is structured in four two-week sprints plus a two-week production hardening period."
- "Show a VISUAL GANTT CHART here"

Phase 1: Sprint 1-2 (Days 1-14) — Foundation & Security Baseline

- Stand up GovCloud environment via Terraform
- Implement Cognito integration with CMS NEE (National External Entity) identity
- KMS key provisioning, DynamoDB table creation
- CI/CD pipeline with automated tests
- **Deliverable:** Secure dev environment, IaC baseline, CI/CD running
- **Risk mitigation:** If NEE integration details are incomplete, we proceed with Cognito native and swap the IdP later. The application layer is IdP-agnostic.

Phase 2: Sprint 3-4 (Days 15-28) — Core Routing & HealthCare.gov Integration

- Step Functions routing engine with production timeouts (15 min)
- Consumer intake widget embedded in HealthCare.gov test environment
- Agent dashboard with real-time WebSocket
- SMS (SNS) and email (SES) notification pipelines
- **Deliverable:** End-to-end routing working in dev, consumer form integrated with test HealthCare.gov
- **Risk mitigation:** If HealthCare.gov API specs are unavailable, we build against our documented API contract and adjust during integration testing.

Phase 3: Sprint 5-6 (Days 29-42) — Admin Operations & Compliance

- Admin dashboard with live metrics, compliance flags, business rule configuration
- Safety-net broadcast implementation
- Missed-referral tracking and deactivation rules
- Agent profile sync with MLMS (license states, languages, training status)
- **Deliverable:** Full admin operational capability, compliance automation
- **Risk mitigation:** If MLMS API access is delayed, we use batch file import as a fallback and switch to API when available.

Phase 4: Sprint 7-8 (Days 43-56) — Load Testing & ATO Preparation

- Load testing against 25k concurrent consumers / 2.5k agents
- Security scanning (static + dynamic)
- ATO evidence collection: architecture docs, security controls, test results
- Penetration testing remediation
- **Deliverable:** Load test report, ATO package submitted

Phase 5: Days 57-90 — Production Hardening & Cutover

- Production deployment to GovCloud
- Parallel run with existing HOD system
- Stakeholder UAT sign-off
- Cutover and HOD decommission
- **Deliverable:** Production live, HOD retired, 30-day post-launch support
- "Key principle: we demo working software at the end of every sprint. No big-bang reveal. The Government sees progress every two weeks."
- "If required artifacts from CMS are incomplete (API specs, MLMS access, NEE integration), we build against documented assumptions and adjust. The prototype proves every component works"

today."

Live Demo Actions

1. Show the deploy script (GitHub: [scripts/deploy.sh](#))

- "This is the one-command deployment. In production, this runs in CI/CD with the GovCloud region parameter."

2. Show Terraform outputs (terminal: `terraform output`)

- "Every endpoint, table, and resource is outputs. The Government can inspect the full deployment at any time."

Transition

"Scalability is built into the architecture. Sheyn will explain."

Q6: Scalability (7 min) — Sheyn

Talking Points

- "The architecture is 100% serverless. No servers to provision, no auto-scaling groups to tune. Every component scales horizontally by default."
- "DynamoDB on-demand capacity: handles 25k concurrent consumers without capacity planning. Single-digit millisecond latency at any scale."
- "Lambda: concurrent executions scale automatically. 1,000+ requests per second is well within Lambda's default concurrency."
- "Step Functions: handles unlimited concurrent state machine executions. Each consumer request is an independent execution."
- "API Gateway: manages connection pooling, throttling, and WebSocket connections natively. 10,000+ concurrent WebSocket connections per API."
- "CloudFront: global CDN for the frontend. Cached at edge locations. Consumers get sub-100ms page loads."
- "Load testing plan: use Artillery or k6 to simulate 25k concurrent consumer submissions and 2.5k agents with active WebSocket connections. Measure: API latency p99 < 500ms, WebSocket push latency < 1s, Step Functions execution time < 5s, zero dropped notifications."
- "Monitoring: CloudWatch dashboards for Lambda errors, DynamoDB throttles, Step Functions failures, API Gateway 4xx/5xx rates. Alarms on error rates and latency thresholds. The admin dashboard itself is a real-time ops console."
- "The concurrency targets from the SOO (25k consumers / 2.5k agents / 5 admins) are surfaced in the admin capacity panel."

Live Demo Actions

1. Show the admin dashboard capacity panel (Window: admin)

- Point to the capacity numbers: "25,000 consumers, 2,500 agents, 5 admins"

- "These are the SOO targets. The serverless architecture scales to meet them without any infrastructure changes."
- 2. **Show DynamoDB table config** (GitHub: [infra/terraform/dynamodb.tf](#))
- "On-demand billing mode. No capacity planning. Pay per request."
- 3. **Show Step Functions definition** (GitHub: [infra/terraform/stepfunctions.tf](#))
- "Each consumer request gets its own execution. No shared state, no contention. Scales infinitely."
- 4. **Show the admin metrics** (Window: admin)
- "These metrics update in real-time via WebSocket. In production, CloudWatch alarms would page on-call for anomalies."

Transition

"Finally, let's talk about the team that will run this. Bob will close us out."

Q7: Staffing Plan (7 min) — Bob

Talking Points

- "Our team covers every SOO requirement: technical leadership, full-stack development, DevSecOps, and program management."
- "I'm Bob, the Program Manager. My role is sprint planning, stakeholder communication, status reporting, and risk management. I'm the single point of contact for CMS."

Day-to-day project management:

- "Two-week sprints with demo at the end of each. Sprint board in GitHub Projects. Weekly status report to CMS COR. Daily standup among the technical team."
- "I maintain the risk register, action item log, and delivery milestones. All visible to the Government in real-time via the GitHub repo."

Integration support:

- "Sheyn owns the HealthCare.gov integration. He's the technical interface to the CMS Marketplace integration team. Nathan implements the API contracts and consumer widget embedding."

Training & help-desk operations:

- "We build admin training materials alongside the software. The admin dashboard is designed to be self-explanatory, but we provide: video walkthroughs, written runbooks for common operations (adding agents, adjusting business rules, handling compliance flags), and a 30-day post-launch support period."
- "For agents/brokers: the dashboard has inline hints, tooltips, and a training portal link. Agent onboarding is self-service via Cognito."

Surge support during Open Enrollment:

- "The serverless architecture means no infrastructure scaling is needed during surge. But for operational surge: Sheyn and Nathan are on-call during OE peak weeks. Sesham monitors security and performance dashboards daily. I increase to daily check-ins with CMS during OE."
- "If additional engineering capacity is needed, our Terraform-based deployment means a new engineer can be onboarded in days, not weeks. The codebase is clean, documented, and the deploy script reproduces the full environment."

Staffing summary:

Role	Name	Responsibility
Tech Lead	Sheyn	Architecture, routing logic, integrations, code review
Full Stack Engineer	Nathan	Frontend + backend development, testing, bug fixes
DevSecOps	Sesham	Infrastructure, security, CI/CD, monitoring, ATO support
Program Manager	Bob	Sprint planning, stakeholder communication, reporting, risk management, training

Live Demo Actions

1. **Show the GitHub project board** (if set up) or the commit history
 - "Every commit is visible. The Government can see exactly who did what, when. Full transparency."
2. **Show the admin compliance panel** (Window: admin)
 - "This is the operational tool a CMS admin would use day-to-day. Compliance flags for missed referrals, stale passwords, lapsed training. All automated."

Closing (2 min) — Bob

"To summarize: we've demonstrated a fully working, non-proprietary prototype that's deployed on AWS today. Every line of code is in our public GitHub repo. The routing engine, business rules, security controls, and admin operations are all live and functional. Our 90-day plan takes this prototype to production in GovCloud with ATO. And our team has the skills and availability to deliver it. We're happy to take questions."

Backup: If Live Demo Fails

- **WebSocket push doesn't arrive:** Refresh the agent dashboard page. The WebSocket reconnects automatically.
- **Request times out before accept:** The demo timeout is 30 seconds. If you need more time, change it in the admin dashboard Business Rules card before the demo.
- **CloudFront cache:** Hard-refresh with Cmd+Shift+R. Or invalidate via `aws cloudfront create-invalidation --distribution-id ESERW07V1HRCA --paths "/*"`.

- **Total failure:** The GitHub repo has the architecture.md with Mermaid diagrams that tell the full story without a live demo. Fall back to walking through the code.
-

Quick Reference: Demo Credentials

Role	Email	Password
Agent (demo)	agent@ran.demo	Agent#Demo123
Agent (Bob Amos, MI/French)	bob.amos@ran.demo	Agent#Demo123
Agent (Kevin McTigue, MI/German)	kevin.mctigue@ran.demo	Agent#Demo123
Admin	admin@ran.demo	Admin#Demo123

Quick Reference: Demo Routing

ZIP	State	Language	Routes To
48201	MI	French	Bob Amos (only MI agent with French)
48201	MI	German	Kevin McTigue (only MI agent with German)
48201	MI	English	Either Bob or Kevin (load-balanced)
331xx	FL	English	Any of 6 FL agents (standard routing)